

uniApp 官方控件基本使用方法（Vue3 setup 版）

说明：uniApp 官方控件是跨端兼容的基础组件，无需额外安装，直接在页面中使用即可，以下所有示例均适配 Vue3 setup 语法，兼顾多端（小程序、H5、App）适配，重点讲解高频常用控件。

一、核心基础控件（必学）

此类控件是开发中最常用、最基础的组件，覆盖页面布局、文本、按钮等核心场景，所有控件均支持 uniApp 官方属性，可根据需求灵活配置。

1. view 控件（布局容器）

作用：类似 HTML 中的 div，用于页面布局，承载其他控件，支持 flex、grid 等布局方式，是页面结构的核心容器。

基本使用

```
1  <template>
2    <!-- 基础布局：嵌套使用，实现页面结构 -->
3    <view class="container">
4      <view class="header">页面头部</view>
5      <view class="content">页面内容</view>
6      <view class="footer">页面底部</view>
7    </view>
8  </template>
9
10  <script setup>
11    // 无需额外引入，直接使用
12  </script>
13
14  <style scoped>
15    .container {
16      display: flex;
17      flex-direction: column;
18      min-height: 100vh;
19    }
20    .header {
21      height: 100rpx;
22      background: #409eff;
```

```

23     color: #fff;
24     display: flex;
25     align-items: center;
26     justify-content: center;
27 }
28 .content {
29     flex: 1;
30     padding: 30rpx;
31 }
32 .footer {
33     height: 100rpx;
34     background: #f5f5f5;
35     display: flex;
36     align-items: center;
37     justify-content: center;
38 }
39 </style>

```

关键注意事项

- view 是块级元素，默认独占一行，可通过 `display: inline-flex` 改为行内块级；
- 多端适配：在 App 端、小程序端表现一致，H5 端可正常渲染，无需额外适配；
- 支持所有 CSS 样式，可结合 flex、grid 实现复杂布局，推荐用于页面整体结构搭建。

2. text 控件（文本显示）

作用：用于显示文本内容，替代 HTML 中的 span，支持文本换行、加粗、颜色等样式，是唯一能选中复制文本的控件（view 中的文本无法直接选中）。

基本使用

```

1  <template>
2    <view class="text-container">
3      <!-- 基础文本 -->
4      <text class="basic-text">普通文本内容</text>
5
6      <!-- 可选中复制的文本 -->
7      <text class="select-text" selectable>可选中复制的文本</text>
8
9      <!-- 换行文本（需配合 white-space: pre-wrap） -->
10     <text class="wrap-text">这是一段需要换行的文本，很长很长很长很长很长很长很长很长
        很长很长很长很长</text>
11
12     <!-- 带删除线的文本 -->

```

```
13     <text class="delete-text" decode>价格: &yen;99.00</text>
14   </view>
15 </template>
16
17 <script setup></script>
18
19 <style scoped>
20   .basic-text {
21     font-size: 32rpx;
22     color: #333;
23     margin-bottom: 20rpx;
24     display: block;
25   }
26   .select-text {
27     font-size: 30rpx;
28     color: #409eff;
29     margin-bottom: 20rpx;
30     display: block;
31   }
32   .wrap-text {
33     font-size: 28rpx;
34     color: #666;
35     white-space: pre-wrap; /* 自动换行 */
36     margin-bottom: 20rpx;
37     display: block;
38   }
39   .delete-text {
40     font-size: 30rpx;
41     text-decoration: line-through;
42     color: #999;
43   }
44 </style>
```

关键注意事项

- text 是行内元素，默认不换行，需通过 `white-space: pre-wrap` 或 `display: block` 实现换行；
- selectable 属性：设置为 true 时，文本可选中复制（仅 text 控件支持）；
- decode 属性：用于解析 HTML 实体（如 ¥ 解析为 ¥），不设置则会原样显示实体符号；
- 避免嵌套：text 控件内尽量只放文本，不嵌套 view 等其他控件，否则可能导致多端适配异常。

3. button 控件（按钮）

作用：用于触发交互操作（如提交、跳转、删除等），自带多端适配样式，支持多种类型、尺寸，可自定义样式。

基本使用

```
1  <template>
2    <view class="btn-container">
3      <!-- 基础按钮（默认类型） -->
4      <button @click="handleClick">默认按钮</button>
5
6      <!-- 主要按钮（primary） -->
7      <button type="primary" @click="handleClick">主要按钮</button>
8
9      <!-- 警告按钮（warn） -->
10     <button type="warn" @click="handleDelete">删除按钮</button><!-- 禁用按钮 -->
11     <button type="primary" disabled>禁用按钮</button>
12
13     <!-- 带图标的按钮 -->
14     <button type="primary">
15       <uni-icon type="search" size="24" color="#fff"></uni-icon>
16       搜索按钮
17     </button>
18
19     <!-- 自定义尺寸按钮 -->
20     <button type="primary" style="width: 200rpx; height: 80rpx;">小按钮
21   </button>
22 </view>
23 </template>
24
25 <script setup>
26   const handleClick = () => {
27     uni.showToast({ title: '按钮点击成功', icon: 'success' })
28   }
29
30   const handleDelete = () => {
31     uni.showModal({
32       title: '提示',
33       content: '确定要删除吗？',
34       success: (res) => {
35         if (res.confirm) {
36           uni.showToast({ title: '删除成功', icon: 'success' })
37         }
38       }
39     })
40   }
41 </script>
```

```
41
42 <style scoped>
43 .btn-container {
44   padding: 30rpx;
45   display: flex;
46   flex-direction: column;
47   gap: 20rpx;
48 }
49 </style>
```

关键注意事项

- type 属性：支持 primary（蓝色）、default（灰色）、warn（红色）三种默认类型，可自定义背景色覆盖；
- 禁用状态：设置 disabled="true" 即可禁用按钮，禁用后点击事件不生效；
- 图标搭配：可嵌套 uni-icon 控件实现带图标按钮，注意图标与文本的间距适配；
- 多端差异：App 端、小程序端按钮样式统一，H5 端可通过 CSS 自定义调整，避免使用过大尺寸导致变形。

二、高频交互控件

此类控件用于实现用户交互（输入、选择、弹窗等），是表单、操作类页面的核心，重点讲解用法和交互逻辑。

1. input 控件（输入框）

作用：用于接收用户输入（如账号、密码、搜索内容等），支持文本、数字、密码等多种输入类型，可配合表单验证使用。

基本使用

```
1 <template>
2   <view class="input-container"><!-- 普通文本输入 -->
3     <view class="input-item">
4       <text class="label">账号：</text>
5       <input
6         type="text"
7         v-model="username"
8         placeholder="请输入账号"
9         @input="handleInput"
10      />
11     </view>
```

```
12
13     <!-- 密码输入 -->
14     <view class="input-item">
15         <text class="label">密码: </text>
16         <input
17             type="password"
18             v-model="password"
19             placeholder="请输入密码"
20         />
21     </view>
22
23     <!-- 数字输入 -->
24     <view class="input-item">
25         <text class="label">手机号: </text>
26         <input
27             type="number"
28             v-model="phone"
29             placeholder="请输入手机号"
30             maxLength="11"
31         />
32     </view>
33
34     <!-- 带清除按钮的输入框 -->
35     <view class="input-item">
36         <input
37             type="text"
38             v-model="search"
39             placeholder="请输入搜索内容"
40             clearable
41         />
42     </view>
43 </view>
44 </template>
45
46 <script setup>
47 import { ref } from 'vue'
48
49 // 响应式输入值
50 const username = ref('')
51 const password = ref('')
52 const phone = ref('')
53 const search = ref('')
54
55 // 输入监听
56 const handleInput = (e) => {
57     console.log('账号输入: ', e.detail.value)
58 }
```

```
59 </script>
60
61 <style scoped>
62   .input-container {
63     padding: 30rpx;
64   }
65   .input-item {
66     display: flex;
67     align-items: center;
68     margin-bottom: 30rpx;
69     border-bottom: 1rpx solid #eee;
70     padding-bottom: 10rpx;
71   }
72   .label {
73     font-size: 30rpx;
74     color: #333;
75     width: 120rpx;
76   }
77   input {
78     flex: 1;
79     font-size: 30rpx;
80     padding: 10rpx 0;
81   }
82   input::placeholder {
83     color: #999;
84     font-size: 28rpx;
85   }
86 </style>
```

关键注意事项

- v-model 绑定：Vue3 中直接用 v-model 绑定响应式数据，无需手动处理 input 事件（可监听 @input 做额外处理）；
- type 属性：支持 text（默认）、password、number、digit（带小数点的数字）、idcard（身份证）等类型；
- clearable 属性：设置为 true 时，输入框右侧会显示清除按钮，点击可清空输入内容；
- maxlength 属性：限制输入最大长度，适用于手机号、密码等场景；
- 多端适配：H5 端 input 样式可通过 CSS 自定义，小程序端、App 端默认样式统一，避免修改默认边框导致适配异常。

2. checkbox / checkbox-group 控件（复选框）

作用：用于用户选择多个选项（如兴趣爱好、协议同意等），checkbox 需嵌套在 checkbox-group 中使用，实现多选逻辑。

基本使用

```
1  <template>
2    <view class="checkbox-container">
3      <text class="title">请选择兴趣爱好：</text>
4      <checkbox-group v-model="checkedList" @change="handleChange">
5        <view class="checkbox-item" v-for="(item, index) in hobbyList"
        :key="index">
6          <checkbox :value="item.value" :checked="item.checked" />
7          <text class="label">{{ item.name }}</text>
8        </view>
9      </checkbox-group>
10
11    <view class="result">已选: {{ checkedList.join(', ') }}</view>
12  </view>
13 </template>
14
15 <script setup>
16 import { ref } from 'vue'
17
18 // 已选列表（v-model 绑定）
19 const checkedList = ref([])
20
21 // 兴趣爱好列表
22 const hobbyList = ref([
23   { name: '篮球', value: 'basketball', checked: false },
24   { name: '足球', value: 'football', checked: false },
25   { name: '游戏', value: 'game', checked: false },
26   { name: '看书', value: 'book', checked: false }
27 ])
28
29 // 选择变化监听
30 const handleChange = (e) => {
31   console.log('已选列表: ', e.detail.value)
32   checkedList.value = e.detail.value
33 }
34 </script>
35
36 <style scoped>
37 .checkbox-container {
38   padding: 30rpx;
39 }
40 .title {
```



```
41     font-size: 32rpx;
42     color: #333;
43     margin-bottom: 30rpx;
44     display: block;
45 }
46 .checkbox-item {
47     display: flex;
48     align-items: center;
49     margin-bottom: 20rpx;
50     font-size: 30rpx;
51     color: #666;
52 }
53 .checkbox-item checkbox {
54     margin-right: 20rpx;
55 }
56 .result {
57     margin-top: 30rpx;
58     font-size: 30rpx;
59     color: #409eff;
60 }
61 </style>
```

关键注意事项

- 嵌套关系：checkbox 必须放在 checkbox-group 内部，否则无法实现多选联动；
- v-model 绑定：checkbox-group 的 v-model 绑定的是“已选 value 的数组”，与单个 checkbox 的 value 对应；
- checked 属性：单个 checkbox 的 checked 可用于默认选中，也可通过 v-model 间接控制；
- change 事件：checkbox-group 的 @change 事件可监听选择变化，通过 e.detail.value 获取已选列表。

3. radio / radio-group 控件（单选框）

作用：用于用户选择单个选项（如性别、学历等），与 checkbox 类似，radio 需嵌套在 radio-group 中使用，实现单选逻辑。

基本使用

```
1 <template>
2   <view class="radio-container">
3     <text class="title">请选择性别：</text>
4     <radio-group v-model="selectedGender" @change="handleChange">
```

```
5      <view class="radio-item" v-for="(item, index) in genderList"
:key="index">
6          <radio :value="item.value" />
7          <text class="label">{{ item.name }}</text>
8      </view>
9  </radio-group>
10
11      <view class="result">已选: {{ selectedGender === 'male' ? '男' : '女' }}
</view>
12  </view>
13 </template>
14
15 <script setup>
16 import { ref } from 'vue'
17
18 // 已选值 (v-model 绑定)
19 const selectedGender = ref('male') // 默认选中男
20
21 // 性别列表
22 const genderList = ref([
23   { name: '男', value: 'male' },
24   { name: '女', value: 'female' }
25 ])
26
27 // 选择变化监听
28 const handleChange = (e) => {
29   console.log('已选性别: ', e.detail.value)
30   selectedGender.value = e.detail.value
31 }
32 </script>
33
34 <style scoped>
35 .radio-container {
36   padding: 30rpx;
37 }
38 .title {
39   font-size: 32rpx;
40   color: #333;
41   margin-bottom: 30rpx;
42   display: block;
43 }
44 .radio-item {
45   display: flex;
46   align-items: center;
47   margin-bottom: 20rpx;
48   font-size: 30rpx;
49   color: #666;
```

```
50  }
51  .radio-item radio {
52    margin-right: 20rpx;
53  }
54  .result {
55    margin-top: 30rpx;
56    font-size: 30rpx;
57    color: #409eff;
58  }
59  </style>
```

关键注意事项

- 单选逻辑：radio-group 内的 radio 只能选中一个，选中后自动取消其他选项；
- v-model 绑定：radio-group 的 v-model 绑定的是“单个已选 value”，而非数组（与 checkbox-group 区别）；
- 默认选中：通过给 radio-group 的 v-model 赋值对应 radio 的 value，即可实现默认选中；
- 禁用状态：给 radio 添加 disabled 属性，可禁用该选项，禁用后无法选中。

4. picker 控件（选择器）

作用：用于弹出选择面板，选择日期、时间、普通选项等（如出生日期、地区选择），支持多列选择，多端适配性强。

基本使用（普通选项选择）

```
1  <template>
2    <view class="picker-container">
3      <view class="picker-item" @click="showPicker">
4        <text class="label">请选择学历：</text>
5        <text class="value">{{ selectedEdu || '请选择' }}&lt;/text&gt;
6      &lt;/view&gt;
7
8      <!-- 普通选择器 -->
9      <picker
10        mode="selector"
11        :range="eduList"
12        :value="selectedIndex"
13        @change="handlePickerChange"
14        v-if="showPickerFlag"
15      ></picker>
16    </view>
17  </template>
```

```
18
19 <script setup>
20 import { ref } from 'vue'
21
22 // 学历列表 (range 数据源)
23 const eduList = ref(['小学', '初中', '高中', '大专', '本科', '研究生'])
24 // 已选索引
25 const selectedIndex = ref(0)
26 // 已选值
27 const selectedEdu = ref('')
28 // 控制选择器显示/隐藏
29 const showPickerFlag = ref(false)
30
31 // 显示选择器
32 const showPicker = () => {
33   showPickerFlag.value = true
34 }
35
36 // 选择变化
37 const handlePickerChange = (e) => {
38   selectedIndex.value = e.detail.value
39   selectedEdu.value = eduList.value[e.detail.value]
40   showPickerFlag.value = false // 选择后关闭选择器
41 }
42 </script>
43
44 <style scoped>
45 .picker-container {
46   padding: 30rpx;
47 }
48 .picker-item {
49   display: flex;
50   justify-content: space-between;
51   align-items: center;
52   border-bottom: 1rpx solid #eee;
53   padding: 20rpx 0;
54   font-size: 30rpx;
55 }
56 .label {
57   color: #333;
58 }
59 .value {
60   color: #999;
61 }
62 </style>
```

常用 mode 类型（重点）

- selector（默认）：普通选项选择，需配置 range 数组作为数据源；
- date：日期选择，可配置 start、end 限制日期范围，如 `:start="2000-01-01"`
`:end="2024-12-31"`；
- time：时间选择，可配置 start、end 限制时间范围，如 `:start="09:00"`
`:end="18:00"`；
- multiSelector：多列选择（如地区选择：省-市-区），需配置 range 为二维数组。

关键注意事项

- range 数据源：普通选择器（selector）的 range 是一维数组，多列选择器（multiSelector）的 range 是二维数组；
- value 属性：绑定的是“选中项的索引”，而非选中项的值，需通过索引获取对应的值；
- 显示控制：可通过 v-if 或 v-show 控制 picker 的显示/隐藏，点击其他元素触发显示；
- 多端差异：App 端、小程序端选择器样式统一，H5 端会渲染为原生选择器，无需额外适配。

三、其他常用控件

1. image 控件（图片显示）

作用：用于显示图片（本地图片、网络图片），支持图片预览、缩放、懒加载等功能，是页面中展示图片的核心控件。

```
1  <template>
2    <view class="image-container">
3      <!-- 本地图片（static目录下） -->
4      <image src="/static/logo.png" class="img" mode="widthFix"
5        @click="previewImage">&lt;/image&gt;
6
7      <!-- 网络图片 -->
8      <image
9        src="https://example.com/image.jpg"
10       class="img"
11       mode="aspectFill"
12       lazy-load
13       @error="handleImageError"
14     >&lt;/image>
15    </view>
16  </template>
```

```
17 <script setup>
18 // 图片预览
19 const previewImage = () => {
20   uni.previewImage({
21     urls: ['/static/logo.png'], // 预览图片数组
22     current: 0 // 当前预览索引
23   })
24 }
25
26 // 图片加载失败处理
27 const handleImageError = (e) => {
28   console.log('图片加载失败', e)
29   // 替换为默认图片
30   e.target.src = '/static/default.png'
31 }
32 </script>
33
34 <style scoped>
35 .image-container {
36   padding: 30rpx;
37   display: flex;
38   gap: 20rpx;
39   flex-wrap: wrap;
40 }
41 .img {
42   width: 200rpx;
43   height: 200rpx;
44   border-radius: 10rpx;
45 }
46 </style>
```

关键注意事项

- src 路径：本地图片需放在 static 目录下，路径以 / 开头；网络图片需填写完整 URL，确保跨域可访问；
- mode 属性：控制图片缩放模式（重点），常用值：
 - widthFix：宽度固定，高度自适应（推荐用于图文列表）；
 - aspectFill：保持宽高比，填满容器，超出部分裁剪；
 - aspectFit：保持宽高比，完全显示在容器内，可能留有空白；
- lazy-load 属性：设置为 true 时，图片懒加载（滚动到可视区域才加载），优化页面性能；
- error 事件：图片加载失败时触发，可用于替换默认图片，提升用户体验；

- 预览功能：通过 uni.previewImage 方法，可实现图片预览（支持手势缩放）。

2. uni-icon 控件（图标）

作用：用于显示常用图标（如搜索、返回、删除等），uniApp 内置了大量图标，无需额外引入，可直接使用。

```
1  <template>
2    &lt;view class="icon-container"&gt;
3      <!-- 基础图标 -->
4      <uni-icon type="search" size="32" color="#409eff"></uni-icon>
5
6      <!-- 带点击事件的图标 -->
7      <uni-icon type="back" size="36" color="#333" @click="goBack"></uni-icon>
8
9      <!-- 警告图标 -->
10     <uni-icon type="warn" size="40" color="#f56c6c"&gt;&lt;/uni-icon&gt;
11
12     <!-- 自定义图标（需配合字体图标） -->
13     <uni-icon type="custom" size="32" color="#67c23a"></uni-icon>
14   </view>
15 </template>
16
17 <script setup>
18   const goBack = () => {
19     uni.navigateBack()
20   }
21 </script>
22
23 <style scoped>
24   .icon-container {
25     padding: 30rpx;
26     display: flex;
27     gap: 30rpx;
28     align-items: center;
29   }
30 </style>
```

关键注意事项

- type 属性：内置图标类型（如 search、back、warn、success、delete 等），可参考 uniApp 官方文档查询所有内置图标；
- size 属性：控制图标大小，单位为 px（无需写 rpx，uniApp 会自动适配多端）；
- color 属性：控制图标颜色，支持十六进制、rgb 等颜色格式；

- 自定义图标：若内置图标不够用，可引入第三方字体图标（如 iconfont），配合 `type="custom"` 使用。

四、通用注意事项（所有控件通用）

1. 无需引入：所有 uniApp 官方控件均无需 import 引入，直接在 template 中使用即可；
2. 多端适配：官方控件已做好多端兼容，尽量使用控件自带属性，避免自定义样式导致多端显示异常；
3. 事件绑定：Vue3 setup 中，直接使用 @事件名（如 @click、@change）绑定方法，无需写 .native 修饰符；
4. 样式优先级：控件默认样式可通过内联样式或类名覆盖，注意样式权重，避免样式不生效；
5. 性能优化：图片控件使用 lazy-load，避免一次性加载过多控件，减少页面渲染压力；
6. 官方文档：若需查看更多控件（如 scroll-view、swiper 等）的用法，可参考 uniApp 官方组件文档，获取完整属性和示例。

（注：文档部分内容可能由 AI 生成）